

Skills Test: Survey Data Deduplication via GraphQL

Background

You are working with a survey platform that stores survey responses in a database exposed via a **GraphQL API**. New survey responses are periodically delivered as a file and must be uploaded to the platform. Before inserting new responses, the system must detect and remove duplicates that already exist in the database.

Provide a brief written explanation of your approach, and the logic of your solution.

Task

Build a program that performs the following steps:

1. Read input data

- Read survey responses from a local file (e.g. CSV or JSON)
- Each record represents one survey response and includes a unique identifier (e.g. `response_id`).

2. Authenticate

- Obtain an authentication token from the API using provided credentials.
- Use this token to authorize subsequent GraphQL requests.

3. Check for duplicates

- Query the GraphQL API to determine which survey responses from the file already exist in the database.
- Clearly define what constitutes a “duplicate” (e.g. matching `response_id`).

4. Deduplicate

- Remove any records from the input data that already exist in the database.
- Ensure the remaining dataset contains only new responses.

5. Upload deduplicated data

- Use a GraphQL mutation to upload the deduplicated survey responses to the database.
-

Constraints & Requirements

- You may use **any programming language**.
 - You must interact with the database via **GraphQL queries and mutations**.
 - Authentication must be handled using an API token.
 - Code should be well-structured and readable.
 - Include basic error handling (e.g. failed API calls, malformed input).
 - Provide comments throughout your code to explain your logic and any assumptions. You can include TODOs for anything you haven't had time to do.
 - A brief commentary on to explain how you approached the task
 - You are free to any AI tools or other support, but you must document your approach to using these tools in your explanation.
-

What We're Looking For

Clean, understandable code

- We don't expect you to be familiar with GraphQL, but we'd like understand your interpretation of the examples and documentation given, and how you would approach a tool like this.
- We're looking for a sensible and logical approach to deduplication and error handling
- Clear and documented flow of authentication and handling of API calls
- Good engineering judgment and communication

Explanation

As well as providing comments within the code itself, we expect a brief commentary of your approach to the task.

You can also include any areas you think could be improved upon in future iterations, or any additional features that may be useful.

Logic

We don't expect the code to be fully completed or particularly complex, it's more important to see your approach to the task and understand how you thought through the process.

Sample GraphQL Schema

SurveyResponse Type

```
type SurveyResponse {
  id: UUID!
  responseId: UUID!
  surveyId: String!
  answers: JSON!
  submittedAt: DateTime!
}
```

Queries

```
type Query {
  surveyResponsesByResponseIds (
    responseIds: [UUID!]!
  ): [SurveyResponse!]!
}
```

Description:

Returns existing survey responses matching the provided `responseIds`.

Mutations

```
type Mutation {
  createSurveyResponses (
    input: [CreateSurveyResponseInput!]!
  ): [SurveyResponse!]!
}
```

```
input CreateSurveyResponseInput {
  responseId: UUID!
  surveyId: String!
  answers: JSON!
}
```

```
submittedAt: DateTime!  
}
```

Authentication API (Mock)

Token Endpoint

POST /auth/token

Request Body (JSON)

```
{  
  "client_id": "your-client-id",  
  "client_secret": "your-client-secret"  
}
```

Response

```
{  
  "access_token": "mock-access-token",  
  "expires_in": 3600,  
  "token_type": "Bearer"  
}
```

Usage

Include the token in all GraphQL requests:

Authorization: Bearer mock-access-token

GraphQL API Endpoint (Mock)

POST /graphql

Required Headers

Content-Type: application/json

Authorization: Bearer <token>

How to Call the API for GraphQL Queries and Mutations

Step 1: Authentication

To interact with the GraphQL API, you first need to authenticate by sending a request to the `/auth/token` endpoint. The response will include an access token, which must be used in the `Authorization` header for all subsequent API requests.

Example Authentication Request

```
curl -X POST http://api.yourplatform.com/auth/token \
-H "Content-Type: application/json" \
-d '{
  "client_id": "your-client-id",
  "client_secret": "your-client-secret"
}'
```

Example Response

```
{
  "access_token": "mock-access-token",
  "expires_in": 3600,
  "token_type": "Bearer"
}
```

Step 2: Send GraphQL Query or Mutation

Once you have the authentication token, you can make GraphQL queries or mutations by sending an HTTP POST request to the `/graphql` endpoint.

Example GraphQL Query (Check for Duplicates)

This query checks whether any responses in your dataset already exist in the platform.

```
curl -X POST http://api.yourplatform.com/graphql \
-H "Content-Type: application/json" \
```

```
-H "Authorization: Bearer mock-access-token" \
-d '{
  "query": "query ($ids: [UUID!]!) { surveyResponsesByResponseIds(respc
  "variables": {
    "ids": [
      "123e4567-e89b-12d3-a456-426614174000",
      "987f6543-b21c-34d3-b765-548814924111",
      "b12a56d2-34f7-44d3-8e6d-9f59e2b42d2f"
    ]
  }
}'
```

Example Query Response

```
{
  "data": {
    "surveyResponsesByResponseIds": [
      {
        "responseId": "123e4567-e89b-12d3-a456-426614174000",
        "surveyId": "survey_001",
        "submittedAt": "2025-01-01T12:00:00Z"
      },
      {
        "responseId": "987f6543-b21c-34d3-b765-548814924111",
        "surveyId": "survey_002",
        "submittedAt": "2025-01-02T14:30:00Z"
      }
    ]
  }
}
```

Example GraphQL Mutation (Upload Deduplicated Data)

Once duplicates are removed, you can upload the deduplicated responses using a GraphQL mutation.

```
curl -X POST http://api.yourplatform.com/graphql \
-H "Content-Type: application/json" \
-H "Authorization: Bearer mock-access-token" \
-d '{
  "query": "mutation ($input: [CreateSurveyResponseInput!]!) { createSu
  "variables": {
    "input": [
```

```

    {
      "responseId": "e4f3a2b1-c67d-4f23-9d9a-df74a5e635d8",
      "surveyId": "survey_123",
      "answers": { "q1": "yes", "q2": "no" },
      "submittedAt": "2025-01-01T12:00:00Z"
    }
  ]
}
}'

```

Example Mutation Response

```

{
  "data": {
    "createSurveyResponses": [
      { "responseId": "e4f3a2b1-c67d-4f23-9d9a-df74a5e635d8" }
    ]
  }
}

```

Handling Errors

If an error occurs (e.g., invalid authentication, malformed request, etc.), the API will return an error message.

Example Error Response:

```

{
  "errors": [
    {
      "message": "Argument \"responseIds\" has an invalid value. Expected",
      "locations": [
        {
          "line": 1,
          "column": 3
        }
      ],
      "path": [
        "query",
        "surveyResponsesByResponseIds"
      ]
    }
  ]
}

```

